

Manual Completo - SoeiroTech Engineering Atlas

Guia de arquitetura, desenvolvimento, estudo, operação e portfólio

0 7 / 0 8 / 2 0 2 6

Sumário

- Manual Completo - SoeiroTech Engineering Atlas
 - Como usar este manual
- 1 . Identidade do projeto
 - 1 . 1 Nome
 - 1 . 2 Tipo de projeto
 - 1 . 3 Problema que o projeto resolve
 - 1 . 4 Visão
 - 1 . 5 O que o projeto não é
- 2 . Objetivos
 - 2 . 1 Objetivo principal
 - 2 . 2 Objetivos iniciais
 - 2 . 3 Resultado profissional esperado
- 3 . Escopo
 - 3 . 1 Dentro do MVP
 - 3 . 2 Fora do MVP
 - 3 . 3 Regra de entrada de novas features
- 4 . Estado atual do projeto
 - 4 . 1 Versão
 - 4 . 2 Fundação já criada
 - 4 . 3 Runtime já validado no ambiente
 - 4 . 4 O que ainda não está concluído
- 5 . Arquitetura
 - 5 . 1 Estilo arquitetural inicial
 - 5 . 2 Por que não começar com Next.js + NestJS separados
 - 5 . 3 ADR- 0 0 1
 - 5 . 4 Diretrizes arquiteturais
- 6 . Stack e responsabilidade de cada tecnologia
 - 6 . 1 Dependências declaradas
- 7 . Estrutura do repositório
 - 7 . 1 `src/`
 - 7 . 2 `prisma/`
 - 7 . 3 `docs/`
 - 7 . 4 `study/`
 - 7 . 5 `.github/`

- 7.6 .vscode/
- 8. docs/ versus study/
 - 8.1 docs/ - verdade oficial do produto
 - 8.2 study/ - aprendizado
 - 8.3 Fluxo de promoção
 - 8.4 Regra prática
- 9. Requisitos de ambiente
 - 9.1 Ambiente esperado
 - 9.2 Arquivos de ambiente
 - 9.3 Configuração padrão
- 10. Portas e rede Docker
 - 10.1 Mapeamento
 - 10.2 Conceito importante
 - 10.3 Banco dentro da rede Docker
- 11. Primeira execução
 - 11.1 Criar o .env
 - 11.2 Subir os serviços
 - 11.3 Inspecionar estado
 - 11.4 Logs
 - 11.5 Verificar Prisma
 - 11.6 Migration inicial
 - 11.7 Seed
 - 11.8 URLs principais
- 12. Docker em detalhe
 - 12.1 Serviços do compose.yaml
 - 12.2 Serviço db
 - 12.3 Serviço web
 - 12.4 Atenção: dívida técnica atual do startup
 - 12.5 Dockerfile
- 13. Comandos Docker essenciais
 - Ver serviços
 - Subir
 - Subir reconstruindo
 - Parar sem remover
 - Derrubar containers/rede do Compose
 - Logs
 - Executar comando no container
 - Listar volumes

- 1 4 . Prisma - visão geral
 - 1 4 . 1 O que é Prisma
 - 1 4 . 2 Peças principais
 - 1 4 . 3 Prisma 7 no projeto
- 1 5 . Prisma - ciclo de trabalho
 - 1 5 . 1 Fluxo mental
 - 1 5 . 2 Gerar Client
 - 1 5 . 3 Criar migration
 - 1 5 . 4 Seed
 - 1 5 . 5 Studio
- 1 6 . Modelo de dados
 - 1 6 . 1 Visão geral
 - 1 6 . 2 Enums
- 1 7 . Model Project
 - 1 7 . 1 Campo a campo
 - 1 7 . 2 Anotações Prisma importantes
 - 1 7 . 3 Organização mental
- 1 8 . Models restantes
 - 1 8 . 1 Skill
 - 1 8 . 2 ProjectSkill
 - 1 8 . 3 Evidence
 - 1 8 . 4 ArchitectureDecision
- 1 9 . Seed inicial
 - Projeto
 - Skills iniciais
 - Relação projeto-skill
 - Evidência inicial
- 2 0 . Prisma Client compartilhado
- 2 1 . Next.js - estrutura atual
 - 2 1 . 1 App Router
 - 2 1 . 2 layout.tsx
 - 2 1 . 3 globals.css
 - 2 1 . 4 StatCard
 - 2 1 . 5 Próxima rota planejada
- 2 2 . Health check
- 2 3 . Testes
 - 2 3 . 1 Estratégia
 - 2 3 . 2 Vitest

- 2 3 . 3 Playwright
- 2 3 . 4 Teste de regressão
- 2 4 . Qualidade de código
 - 2 4 . 1 Lint
 - 2 4 . 2 Typecheck
 - 2 4 . 3 Formatação
 - 2 4 . 4 Build
 - 2 4 . 5 Sequência de validação recomendada
- 2 5 . CI com GitHub Actions
- 2 6 . VS Code profissional
 - 2 6 . 1 settings.json
 - 2 6 . 2 Extensões recomendadas
 - 2 6 . 3 Tasks
- 2 7 . Snippets
 - stamp
 - adr
 - devlog
 - nsc
 - zschema
 - vtest
- 2 8 . Agentes de IA
 - 2 8 . 1 Arquiteto
 - 2 8 . 2 Implementador
 - 2 8 . 3 Revisor
 - 2 8 . 4 Mentor
- 2 9 . Prompts reutilizáveis
 - Exemplo de fluxo
- 3 0 . AGENTS.md
 - Definition of Done
- 3 1 . Documentação oficial
 - docs/00-product/
 - docs/01-requirements/
 - docs/02-architecture/
 - docs/03-data/
 - docs/04-decisions/
 - docs/05-quality/
 - docs/06-project/
 - docs/07-devlog/
 - docs/08-showcase/
 - docs/reference/

- **3 2 . Sistema de estudo**
 - **3 2 . 1** 00-inbox
 - **3 2 . 2** 01-concepts
 - **3 2 . 3** 02-labs
 - **3 2 . 4** 03-debugging
 - **3 2 . 5** 04-reviews
 - **3 2 . 6** 05-glossary
 - **3 2 . 7** 06-applied-learning
- **3 3 . Como estudar sem transformar o projeto em apostila**
 - Errado
 - Correto
 - Regra de ouro
- **3 4 . Fluxo de desenvolvimento profissional**
 - **3 4 . 1** Limite de escopo
- **3 5 . Git e GitHub**
 - **3 5 . 1** Estratégia de visibilidade
 - **3 5 . 2** O que mostrar publicamente
 - **3 5 . 3** Commits
- **3 6 . Segurança**
 - **3 6 . 1** Segredos
 - **3 6 . 2** Autorização
 - **3 6 . 3** Validação
 - **3 6 . 4** Dados do showcase
- **3 7 . Riscos do projeto**
- **3 8 . Troubleshooting - Prisma não encontrado**
 - Sintoma
 - Significado
 - Causas possíveis
 - Diagnóstico
- **3 9 . Troubleshooting - npm ENOEMPTY**
 - Sintoma encontrado
 - Abordagem segura
- **4 0 . Troubleshooting - Docker credential helper**
 - Sintoma encontrado
 - Diagnóstico que isolou o problema
 - Aprendizado de debugging
- **4 1 . Troubleshooting - porta ocupada**
 - Situação

- Solução do Atlas
- Verificar
- .env
- 4 2 . Troubleshooting - service web is not running
- 4 3 . Troubleshooting - health degradado
- 4 4 . Acesso direto ao PostgreSQL
- 4 5 . Roadmap
 - Fase 0 - Fundação
 - Fase 1 - Projetos
 - Fase 2 - Skills
 - Fase 3 - Evidências
 - Fase 4 - Decisões
 - Fase 5 - Showcase
- 4 6 . Ponto atual do aprendizado
- 4 7 . Plano de aprendizado por tecnologia
 - Prisma
 - PostgreSQL
 - Next.js
 - TypeScript
 - Docker
- 4 8 . Rotina de trabalho recomendada
 - Ao iniciar uma sessão
 - Durante
 - Antes de encerrar
- 4 9 . Checklist antes de criar uma feature
- 5 0 . Checklist de mudança de banco
- 5 1 . Checklist de Pull Request
- 5 2 . Checklist de evidência profissional
- 5 3 . Como transformar o projeto em case para emprego
 - Estrutura de apresentação
- 5 4 . O que não prometer em entrevista
- 5 5 . Melhorias técnicas recomendadas para a fundação
- 5 6 . Glossário
 - ADR
 - API
 - Build
 - CI
 - Container

- CUID
- Docker Compose
- E 2 E
- Enum
- Foreign Key
- Health Check
- Índice
- Migration
- Modular Monolith
- ORM
- Primary Key
- Prisma Client
- Route Handler
- Seed
- Server Component
- Slug
- Typecheck
- Volume
- **5 7 . Referência rápida de comandos**
 - Docker
 - Prisma dentro do container
 - Qualidade
 - PostgreSQL
 - Git - fluxo básico
- **5 8 . Ficha rápida do projeto**
- **5 9 . Próxima etapa oficial**
- **6 0 . Encerramento**
 - Anexo A - Carimbo Digital
 - Anexo B - Modelo de ADR
 - Anexo C - Modelo de aprendizado aplicado
 - Anexo D - Regra visual de evolução

Versão do projeto documentada: 0 . 1 . 0

Data desta edição: 0 7 / 0 8 / 2 0 2 6

Status: Fundação técnica concluída; domínio em construção

Stack-base: TypeScript, Next.js, React, PostgreSQL, Prisma, Docker

 Painel de Sinais Organizados (Compacto)

 [PJ]  [DEV]  [STUDY]  [CODE]  [DB]  [TEST]  [DOC]  [SEC]

Princípio central: Estudo fornece conhecimento. Projeto fornece contexto. Evidência demonstra competência.

Como usar este manual

Este manual é a referência geral do SoeiroTech Engineering Atlas. Ele foi escrito para servir ao mesmo tempo como:

- guia de instalação e execução;
- mapa da arquitetura;
- explicação didática da stack;
- catálogo dos arquivos e responsabilidades;
- manual de uso do Docker, Prisma e Next.js;
- referência dos agentes, prompts e snippets do VS Code;
- guia para organizar `docs/` e `study/`;
- roteiro de desenvolvimento incremental;
- checklist de qualidade;
- material de apoio para GitHub, showcase e entrevistas;
- registro de problemas já encontrados e de como diagnosticá-los.

Não é necessário ler tudo antes de programar. Use o documento como um mapa consultável. A sequência recomendada é:

- 1 . leia os capítulos 1 a 5 para entender o produto;
- 2 . use os capítulos 6 a 10 quando estiver configurando o ambiente;
- 3 . consulte os capítulos de Prisma, banco, Next.js e testes durante a implementação;
- 4 . use `study/` para aprendizado e `docs/` para decisões oficiais;
- 5 . ao finalizar uma feature, consulte a Definition of Done e o checklist de evidências.

Regra de segurança: não execute comandos destrutivos de Docker, banco ou Git sem entender o efeito. Quando houver dúvida, primeiro inspecione o estado atual.

1 . Identidade do projeto

1 . 1 Nome

SoeiroTech Engineering Atlas

Nome técnico do pacote/repositório:

```
soeirotech-engineering-atlas
```

1 . 2 Tipo de projeto

O Engineering Atlas é um projeto de aprendizado aplicado + portfólio profissional. Ele não foi criado para reproduzir aulas, disciplinas ou apostilas.

O produto deve demonstrar que um desenvolvedor consegue relacionar:

```
Projeto
↓
Problema real
↓
Skill aplicada
↓
Decisão técnica
↓
Implementação
↓
Teste
↓
Evidência
```

1 . 3 Problema que o projeto resolve

Cursos, certificados, anotações e listas de tecnologias mostram exposição a conhecimento, mas nem sempre demonstram onde o conhecimento foi aplicado.

Um desenvolvedor pode possuir vários repositórios e estudos, mas ainda ter dificuldade para responder perguntas como:

- Onde usei PostgreSQL de verdade?
- Que projeto demonstra que sei trabalhar com Docker?
- Qual teste comprova determinada regra?
- Por que escolhi uma arquitetura e não outra?
- Em qual contexto apliquei uma determinada skill?

- Que evidência posso mostrar para um recrutador?

O Engineering Atlas busca criar essa rastreabilidade.

1 . 4 Visão

Construir uma plataforma em que projetos, habilidades, decisões e evidências possam ser conectados de forma clara e verificável.

1 . 5 O que o projeto não é

O projeto não deve se transformar em:

- cópia da grade de uma pós-graduação;
- repositório de aulas inteiras;
- coleção de tecnologias sem necessidade;
- laboratório de microserviços apenas por moda;
- sistema cheio de features antes de validar o domínio central;
- demonstração artificial de padrões de projeto.

A formação acadêmica pode ensinar um conceito. O Atlas só deve incorporar esse conceito quando existir um problema de produto que justifique sua aplicação.

2 . Objetivos

2 . 1 Objetivo principal

Permitir que um desenvolvedor relacione habilidades a projetos e a evidências reais de aplicação.

2 . 2 Objetivos iniciais

O projeto foi definido para evoluir em torno de cinco capacidades principais:

- 1 . Projetos - registrar projetos reais e seu estado.
- 2 . Skills - catalogar habilidades técnicas.
- 3 . Evidências - registrar commits, testes, ADRs, diagramas, screenshots, releases e deploys.
- 4 . Decisões arquiteturais - explicar contexto, escolha e trade-offs.
- 5 . Showcase - apresentar uma parte pública e profissional do trabalho.

2 . 3 Resultado profissional esperado

O sistema deve ajudar a produzir respostas demonstráveis, por exemplo:

“Usei PostgreSQL no projeto X, para modelar Y, com migrations Z e testes relacionados.”

Isso é mais forte profissionalmente do que apenas declarar:

“Conheço PostgreSQL.”

3 . Escopo

3 . 1 Dentro do MVP

O escopo inicial inclui:

- projetos;
- skills;
- relação entre projeto e skill;
- evidências;
- decisões arquiteturais;
- showcase público;
- health check.

3 . 2 Fora do MVP

Estão deliberadamente fora do primeiro MVP:

- rede social;
- marketplace;
- pagamentos;
- gamificação;
- IA generativa integrada ao produto;
- notificações;

- aplicativo mobile;
- microserviços;
- analytics avançado.

A exclusão desses itens não significa que nunca serão feitos. Significa que **não há** justificativa suficiente para adicioná-los agora.

3.3 Regra de entrada de novas features

Antes de implementar uma nova feature, responder:

1. Qual problema resolve?
2. Quem usa?
3. Que dado ou regra exige?
4. Como será validada?
5. Qual evidência profissional produzirá?
6. O que ficará fora do escopo desta entrega?

Se essas respostas não estiverem claras, a feature ainda não está pronta para implementação.

4. Estado atual do projeto

4.1 Versão

O `package.json` define:

```
version: 0.1.0
```

4.2 Fundação já criada

A Fase **0** foi preparada com:

- Next.js;
- TypeScript;
- React;
- PostgreSQL;
- Prisma;

- Docker Compose;
- endpoint de health check;
- Vitest;
- Playwright;
- ESLint;
- Prettier;
- GitHub Actions;
- documentação base;
- agentes e prompts para VS Code/Copilot;
- snippets de desenvolvimento;
- rota inicial de showcase.

4 . 3 Runtime já validado no ambiente

Durante a configuração foi confirmado:

```
Prisma          7.9.1
@prisma/client  7.9.1
Node.js         24.19.0
TypeScript      5.9.3
Operating System Linux x64 (container)
```

O banco utilizado pelo Compose é:

```
PostgreSQL 17 Alpine
```

4 . 4 O que ainda não está concluído

A presença de modelos no `schema.prisma` não significa que o CRUD inteiro já exista na interface.

Ainda precisam ser desenvolvidos incrementalmente:

- CRUD real de projetos;
- CRUD de skills;
- relacionamento visual projeto-skill;
- gerenciamento de evidências;
- gerenciamento de ADRs no sistema;
- seleção do conteúdo público do showcase;
- autenticação/autorização, somente quando o produto exigir;
- pipeline de produção/deploy definitivo.

5 . Arquitetura

5 . 1 Estilo arquitetural inicial

A arquitetura escolhida é um Modular Monolith usando Next.js como aplicação full stack.

```
Navegador
  ↓
Next.js
  ↓
Regras / módulos da aplicação
  ↓
Prisma Client
  ↓
PostgreSQL
```

5 . 2 Por que **não** começar com Next.js + NestJS separados

Separar frontend e backend desde o primeiro commit criaria:

- dois ciclos de build;
- dois processos de desenvolvimento;
- contratos HTTP adicionais;
- mais configuração de CORS;
- mais containers;
- mais pontos de falha;
- mais deploys;
- maior custo cognitivo.

No estágio atual, não existe requisito que pague esse custo.

5 . 3 ADR- 0 0 1

A decisão oficial registrada no projeto é:

ADR- 0 0 1 - Iniciar como Modular Monolith

Alternativas consideradas:

- Next.js + NestJS separados desde o início;
- **microserviços.**

A decisão poderá ser reavaliada quando surgir uma necessidade concreta, como múltiplos clientes independentes, escalabilidade operacional separada ou fronteiras de domínio que justifiquem extração.

5.4 Diretrizes arquiteturais

O projeto adota as seguintes diretrizes:

- Server Components por padrão;
- Route Handlers quando um endpoint for necessário;
- Zod para validação de entradas;
- Prisma para persistência;
- PostgreSQL como banco relacional;
- Docker Compose para ambiente local;
- TypeScript estrito;
- testes baseados em risco;
- ADR para decisões arquiteturais relevantes;
- evitar abstrações antes de existir repetição ou necessidade real.

6. Stack e responsabilidade de cada tecnologia

Tecnologia	Papel no projeto
TypeScript	Linguagem principal e tipagem estática
Next.js	Framework full stack, rotas, páginas e APIs
React	Construção da interface
Tailwind CSS	Estilização utilitária
PostgreSQL	Banco de dados relacional
Prisma	Modelagem, migrations e acesso ao banco
@prisma/adapters	Adapter PostgreSQL usado pelo Prisma 7
Zod	Validação de dados
Docker	Empacotamento e isolamento do ambiente

Tecnologia	Papel no projeto
Docker Compose	Orquestração local de web + banco
Vitest	Testes unitários
Playwright	Testes E 2 E
ESLint	Análise estática de código
Prettier	Formatação
GitHub Actions	CI
Markdown	Documentação e estudo

6 . 1 Dependências declaradas

O projeto declara, entre outras:

```
@prisma/adapters-pg ^7.9.1
@prisma/client ^7.9.1
next ^16.0.0
pg ^8.16.0
react ^19.2.0
react-dom ^19.2.0
zod ^4.1.0
```

E ferramentas de desenvolvimento como:

```
prisma ^7.9.1
typescript ^5.9.0
vitest ^3.2.0
@playwright/test ^1.55.0
eslint ^9.0.0
prettier ^3.6.0
```

As versões com ^ representam faixas compatíveis. Um `package-lock.json` deve ser gerado e versionado para tornar instalações futuras mais reproduzíveis.

7 . Estrutura do repositório

Estrutura conceitual:

```
soeirotech-engineering-atlas/
├── .github/
│   ├── agents/
│   ├── instructions/
│   ├── prompts/
│   ├── workflows/
│   ├── copilot-instructions.md
│   └── pull_request_template.md
├── .vscode/
│   ├── settings.json
│   ├── extensions.json
│   ├── tasks.json
│   └── soeirotech.code-snippets
├── docs/
│   ├── 00-product/
│   ├── 01-requirements/
│   ├── 02-architecture/
│   ├── 03-data/
│   ├── 04-decisions/
│   ├── 05-quality/
│   ├── 06-project/
│   ├── 07-devlog/
│   ├── 08-showcase/
│   ├── reference/
│   └── templates/
├── study/
│   ├── 00-inbox/
│   ├── 01-concepts/
│   ├── 02-labs/
│   ├── 03-debugging/
│   ├── 04-reviews/
│   ├── 05-glossary/
│   ├── 06-applied-learning/
│   └── templates/
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
├── src/
│   ├── app/
│   ├── components/
│   ├── generated/
│   └── lib/
├── tests/
│   └── e2e/
├── AGENTS.md
├── compose.yaml
├── Dockerfile
├── prisma.config.ts
├── package.json
├── README.md
└── SETUP.md
```

7 . 1 `src/`

É o produto executável.

7 . 2 `prisma/`

É a fonte de verdade da modelagem de persistência e migrations.

7 . 3 `docs/`

Contém documentação oficial do produto.

7 . 4 `study/`

Contém aprendizado pessoal e experimentação.

7 . 5 `.github/`

Contém automação, instruções, prompts e agentes do repositório.

7 . 6 `.vscode/`

Contém configurações compartilháveis do workspace.

8 . `docs/` versus `study/`

Essa separação é uma das regras mais importantes do projeto.

8 . 1 `docs/` - verdade oficial do produto

Use `docs/` para:

- visão;
- problema;
- objetivos;
- escopo;
- requisitos;
- arquitetura;

- modelo de dados;
- ADRs;
- estratégia de testes;
- riscos;
- roadmap;
- DevLog relevante;
- case study.

8 . 2 `study/` - aprendizado

Use `study/` para:

- conceitos;
- dúvidas;
- explicações;
- erros reais;
- pequenos experimentos;
- revisões;
- glossário;
- aprendizado aplicado.

8 . 3 Fluxo de promoção

```
Dúvida / conceito
  ↓
study/
  ↓
entendimento
  ↓
aplicação no código
  ↓
evidência
  ↓
se virar decisão oficial
  ↓
docs/
```

8 . 4 Regra prática

Estudar Prisma Migrate:

```
study/01-concepts/prisma/
```

Decidir oficialmente uma política de migration:

```
docs/04-decisions/
```

A migration real:

```
prisma/migrations/
```

Essa separação impede que documentação oficial vire um caderno de aula.

9 . Requisitos de ambiente

9 . 1 Ambiente esperado

Para desenvolvimento com Docker:

- Docker Desktop/Engine funcionando;
- Docker Compose;
- Git;
- VS Code recomendado;
- WSL/Linux é compatível com o fluxo atual.

O container fornece Node.js, portanto não é obrigatório depender do Node instalado no host para executar o ambiente Docker.

9 . 2 Arquivos de ambiente

O projeto fornece:

```
.env.example
```

O arquivo real deve ser criado como:

```
.env
```

O `.env` real não deve ser commitado.

9.3 Configuração padrão

```
APP_NAME=SoeiroTech Engineering Atlas
APP_URL=http://localhost:3010
NODE_ENV=development

WEB_PORT=3010
POSTGRES_PORT=5440

POSTGRES_DB=engineering_atlas
POSTGRES_USER=atlas
POSTGRES_PASSWORD=atlas_dev_password

DATABASE_URL=postgresql://atlas:atlas_dev_password@db:5432/engineering_atlas?schema=public
```

A senha acima é uma credencial de desenvolvimento local do exemplo. Produção deve usar segredo próprio e não versionado.

10. Portas e rede Docker

10.1 Mapeamento

O Engineering Atlas foi configurado para evitar colisões com outros projetos.

Serviço	Host	Container
Next.js	3 0 1 0	3 0 0 0
PostgreSQL	5 4 4 0	5 4 3 2

Isso significa:

```
http://localhost:3010
```

chega ao Next.js que escuta a porta 3000 dentro do container.

10.2 Conceito importante

Vários containers podem escutar internamente a porta 3000:

```
Projeto A: 3010 -> 3000
Projeto B: 3020 -> 3000
Projeto C: 3030 -> 3000
```

O conflito ocorre quando duas aplicações tentam ocupar a mesma porta do host.

1 0 . 3 Banco dentro da rede Docker

A aplicação usa:

```
db:5432
```

Não usa `localhost:5440` dentro do container.

`db` é o nome do serviço Docker Compose e funciona como hostname na rede `engineering-atlas`.

1 1 . Primeira execução

1 1 . 1 Criar o `.env`

No Linux/WSL:

```
cp .env.example .env
```

No PowerShell:

```
Copy-Item .env.example .env
```

1 1 . 2 Subir os serviços

```
docker compose up --build -d
```

1 1 . 3 Inspecionar estado

```
docker compose ps
```

O esperado é encontrar algo equivalente a:

```
engineering-atlas-web    ...  0.0.0.0:3010->3000/tcp
engineering-atlas-db     ...  0.0.0.0:5440->5432/tcp
```

1 1.4 Logs

```
docker compose logs -f web
```

Banco:

```
docker compose logs -f db
```

Ctrl + C sai da visualização dos logs; não derruba o container.

1 1.5 Verificar Prisma

```
docker compose exec web npx prisma -v
```

1 1.6 Migration inicial

```
docker compose exec web npm run db:migrate -- --name init
```

1 1.7 Seed

```
docker compose exec web npm run db:seed
```

1 1.8 URLs principais

Aplicação	http://localhost:3010
Showcase	http://localhost:3010/showcase
Health	http://localhost:3010/api/health

1 2 . Docker em detalhe

1 2 . 1 ServiçOs do `compose.yaml`

O Compose define dois serviçOs:

```
db
web
```

1 2 . 2 ServiçO `db`

Responsabilidades:

- executar PostgreSQL `17` Alpine;
- criar banco e usu rio de desenvolvimento;
- persistir dados em volume;
- expor `5440` no host;
- executar health check com `pg_isready`.

Volume:

```
engineering_atlas_pgdata
```

Esse volume cont m os dados do PostgreSQL e n o deve ser removido casualmente.

1 2 . 3 ServiçO `web`

Responsabilidades:

- executar a aplica  o Next.js;
- montar o c digo do host em `/app`;
- usar volume pr prio de `node_modules`;
- depender do banco saud vel;
- expor `3010 -> 3000`;
- executar health check da aplica  o.

Volume de depend ncias:

```
engineering_atlas_node_modules
```

1 2 . 4 Atenção: dívida técnica atual do startup

O `compose.yaml` atual contém:

```
npm install
npm run db:generate
npm run dev
```

executados no startup do container.

Isso foi útil para a fundação, mas **não é** o desenho final ideal. Instalar dependências toda vez que o container inicia pode aumentar o tempo de startup e favorecer inconsistências no volume de `node_modules`.

Refatoração futura recomendada:

```
Docker build
↓
instala dependências
↓
imagem pronta
↓
container inicia
↓
Prisma generate / app
```

Não alterar isso no meio de uma etapa de estudo sem registrar a mudança e validar o ambiente.

1 2 . 5 Dockerfile

O Dockerfile possui estágios:

- `base`;
- `development`;
- `dependencies`;
- `builder`;
- `production`.

O Compose de desenvolvimento usa:

```
target: development
```

O estágio de produção é um scaffold e ainda deve ser validado antes de chamar o projeto de production-ready.

1 3 . Comandos Docker essenciais

Ver serviços

```
docker compose ps
```

Subir

```
docker compose up -d
```

Subir reconstruindo

```
docker compose up --build -d
```

Parar sem remover

```
docker compose stop
```

Derrubar containers/rede do Compose

```
docker compose down
```

Logs

```
docker compose logs -f web
```

Executar comando no container

```
docker compose exec web <comando>
```

Exemplo:

```
docker compose exec web npm run lint
```

Listar volumes

```
docker volume ls
```

Nunca remova um volume por tentativa. Primeiro confirme se é `node_modules` ou banco.

1 4 . Prisma - visão geral

1 4 . 1 O que é Prisma

Prisma é a camada usada pela aplicação TypeScript para modelar e acessar o PostgreSQL.

Não confundir:

```
PostgreSQL = banco de dados
Prisma      = ferramenta/camada de acesso e modelagem
Next.js     = aplicação
```

Fluxo:

```
Next.js
  ↓
Prisma Client
  ↓
PostgreSQL
```

1 4 . 2 Peças principais

Prisma Schema

```
prisma/schema.prisma
```

Define modelos, campos, relações, enums, índices e constraints suportadas.

Prisma Client

Código gerado que permite escrever consultas tipadas em TypeScript.

Prisma Migrate

Cria e aplica mudanças de estrutura no banco.

Prisma Generate

Gera o Prisma Client a partir do schema.

Prisma Studio

Interface visual de desenvolvimento para inspecionar dados.

1 4 . 3 Prisma 7 no projeto

O projeto utiliza `prisma.config.ts` para configurar datasource e migrations:

```
import "dotenv/config";
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
    seed: "tsx prisma/seed.ts",
  },
  datasource: {
    url: env("DATABASE_URL"),
  },
});
```

No `schema.prisma`, o datasource informa apenas o provider:

```
datasource db {
  provider = "postgresql"
}
```

1 5 . Prisma - ciclo de trabalho

1 5 . 1 Fluxo mental

```
model no schema.prisma
  ↓
prisma migrate dev
  ↓
migration SQL
  ↓
PostgreSQL
  ↓
prisma generate
  ↓
Prisma Client
  ↓
TypeScript consulta dados
```

1 5 . 2 Gerar Client

```
docker compose exec web npm run db:generate
```

1 5 . 3 Criar migration

```
docker compose exec web npm run db:migrate -- --name nome-da-migration
```

Exemplo:

```
docker compose exec web npm run db:migrate -- --name init
```

1 5 . 4 Seed

```
docker compose exec web npm run db:seed
```

1 5 . 5 Studio

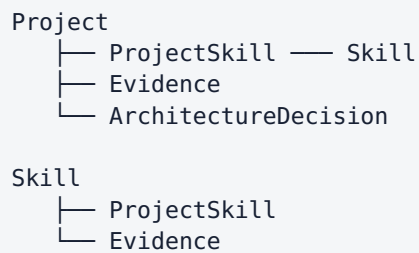
O script existe:

```
npm run db:studio
```

Se executado dentro do container, a porta do Studio precisará ser exposta/configurada para acesso pelo host. Portanto, não presumir que `localhost:5555` estará acessível sem essa configuração.

1 6 . Modelo de dados

1 6 . 1 Visão geral



1 6 . 2 Enums

ProjectStatus

IDEA
PLANNED
IN_PROGRESS
PAUSED
COMPLETED
ARCHIVED

EvidenceType

COMMIT
PULL_REQUEST
TEST
ADR
DIAGRAM
SCREENSHOT
RELEASE
DEPLOY
BENCHMARK
OTHER

DecisionStatus

PROPOSED
ACCEPTED
SUPERSEDED
REJECTED

1 7 . Model **Project**

O model atual é:

```
model Project {  
  id          String          @id @default(cuid())  
  slug        String          @unique  
  name        String  
  description  String?  
  status      ProjectStatus @default(PLANNED)  
  repository  String?  
  demoUrl     String?  
  createdAt   DateTime        @default(now())  
  updatedAt   DateTime        @updatedAt  
  
  skillLinks  ProjectSkill[]  
  evidences   Evidence[]  
  decisions   ArchitectureDecision[]  
  
  @@index([status])  
}
```

1 7 . 1 Campo a campo

Campo	Significado
<code>id</code>	Identificador Único
<code>slug</code>	Identificador amigável para URL e busca
<code>name</code>	Nome do projeto
<code>description</code>	Descrição opcional
<code>status</code>	Estado controlado pelo enum
<code>repository</code>	URL opcional do repositório
<code>demoUrl</code>	URL opcional de demonstração

Campo	Significado
<code>createdAt</code>	Data de criação
<code>updatedAt</code>	Última atualização
<code>skillLinks</code>	Relações com skills
<code>evidences</code>	Evidências do projeto
<code>decisions</code>	Decisões arquiteturais

1 7 . 2 Anotações Prisma importantes

`@id` chave primária
`@unique` impede duplicidade
`@default(...)` valor padrão/gerado
`?` campo opcional
`[]` coleção/relação múltipla
`@@index` índice de modelo

1 7 . 3 Organização mental

```

PROJECT
├── Identidade
│   ├── id
│   ├── slug
│   └── name
├── Informação
│   ├── description
│   ├── repository
│   └── demoUrl
├── Estado
│   └── status
├── Datas
│   ├── createdAt
│   └── updatedAt
└── Relações
    ├── skillLinks
    ├── evidences
    └── decisions

```

1 8 . Models restantes

1 8 . 1 **skill**

Representa uma habilidade técnica, por exemplo:

```
TypeScript  
Next.js  
PostgreSQL  
Prisma  
Docker
```

Possui `slug` Único e pode estar ligada a vários projetos e evidências.

1 8 . 2 **ProjectSkill**

É a entidade de ligação entre projeto e skill.

Além de relacionar os dois lados, possui:

```
context
```

Isso é importante porque saber que um projeto “usa Docker” é menos informativo do que registrar como Docker foi usado.

1 8 . 3 **Evidence**

Representa prova verificável de aplicação.

Campos principais:

- título;
- descrição;
- tipo;
- URL opcional;
- projeto obrigatório;
- skill opcional.

Uma evidência pertence a um projeto e pode, quando fizer sentido, provar uma skill específica.

1 8 . 4 `ArchitectureDecision`

Representa decisão arquitetural ligada a um projeto.

Contém:

- código Único;
- título;
- contexto;
- decisão;
- consequências;
- status;
- projeto.

1 9 . Seed inicial

O `prisma/seed.ts` cria ou garante:

Projeto

```
SoeiroTech Engineering Atlas  
status: IN_PROGRESS
```

Skills iniciais

```
TypeScript  
Next.js  
PostgreSQL  
Prisma  
Docker
```

Relação projeto-skill

Cada skill recebe o contexto:

```
Stack inicial do projeto.
```

Evidência inicial

Fundação do repositório

O uso de `upsert` ajuda a evitar duplicação do projeto e das skills quando o seed é executado novamente.

A evidência inicial usa `create`, portanto executar o seed repetidas vezes pode criar múltiplas evidências equivalentes. Isso é um ponto possível de refatoração futura.

2 0 . Prisma Client compartilhado

Arquivo:

```
src/lib/prisma.ts
```

Responsabilidades:

- 1 . ler `DATABASE_URL` ;
- 2 . criar `PrismaPg` ;
- 3 . criar `PrismaClient` ;
- 4 . reutilizar a instância em desenvolvimento.

O reaproveitamento via `globalThis` ajuda a evitar múltiplas instâncias durante hot reload do ambiente de desenvolvimento.

Fluxo:

```
src/lib/prisma.ts
  ↓
export prisma
  ↓
Route Handler / Server Component / serviço
  ↓
PostgreSQL
```

2 1 . Next.js - estrutura atual

2 1 . 1 App Router

O projeto utiliza `src/app/`.

Mapeamento atual:

```
src/app/page.tsx
  ↓
/
src/app/showcase/page.tsx
  ↓
/showcase
src/app/api/health/route.ts
  ↓
/api/health
```

2 1 . 2 `layout.tsx`

Responsável por layout global, metadata e navegação principal.

2 1 . 3 `globals.css`

Contém estilos globais e variáveis visuais iniciais.

2 1 . 4 `StatCard`

`src/components/stat-card.tsx` é um componente simples reutilizado pela página inicial.

2 1 . 5 Próxima rota planejada

A primeira feature de domínio deve introduzir:

```
/projects
```

Depois, incrementalmente:

```
/projects/new
/projects/[slug ou id]
edição
arquivamento
filtros
```

Não construir todas essas etapas em um único salto.

2 2 . Health check

Endpoint:

```
GET /api/health
```

Arquivo:

```
src/app/api/health/route.ts
```

O endpoint executa uma consulta simples:

```
SELECT 1
```

Se o banco responder, retorna estado saudável. Se falhar, responde com status HTTP **5 0 3** e estado degradado.

Isso verifica duas camadas ao mesmo tempo:

```
Next.js está respondendo
+
PostgreSQL está acessível pelo Prisma
```

É mais útil do que um health check que apenas retorna texto fixo.

2 3 . Testes

2 3 . 1 Estratégia

Prioridade definida:

1 . regras de negócio;

- 2 . autorização;
- 3 . validação;
- 4 . regressões;
- 5 . fluxos críticos.

O projeto não busca cobertura alta por vaidade.

2 3 . 2 Vitest

Comandos:

```
npm test  
npm run test:watch
```

Exemplo já presente:

```
src/lib/slugify.test.ts
```

2 3 . 3 Playwright

Comando:

```
npm run test:e2e
```

Existe um teste inicial para o showcase público.

Para primeira execução local fora de uma imagem já preparada, pode ser necessário instalar o browser do Playwright:

```
npx playwright install chromium
```

2 3 . 4 Teste de regressão

Quando um bug relevante for corrigido, perguntar:

É possível criar um teste que impeça esse bug de voltar?

Se sim, o bug deve ganhar teste de regressão.

2 4 . Qualidade de código

2 4 . 1 Lint

```
npm run lint
```

2 4 . 2 Typecheck

```
npm run typecheck
```

2 4 . 3 Formatação

```
npm run format  
npm run format:check
```

2 4 . 4 Build

```
npm run build
```

2 4 . 5 Sequência de validação recomendada

```
lint  
↓  
typecheck  
↓  
testes  
↓  
build
```

Para uma mudança de banco, acrescentar migration/validação do banco.

2 5 . CI com GitHub Actions

Arquivo:

```
.github/workflows/ci.yml
```

O pipeline definido executa:

- 1 . checkout;
- 2 . setup Node 2 4 ;
- 3 . PostgreSQL de serviço;
- 4 . instalação de dependências;
- 5 . Prisma generate;
- 6 . lint;
- 7 . typecheck;
- 8 . testes unitários;
- 9 . build.

O CI roda em:

```
push para main  
pull request para main
```

O objetivo é impedir que código claramente quebrado seja integrado sem detecção automática.

A pipeline atual é base de qualidade, não um pipeline de deploy completo.

2 6 . VS Code profissional

2 6 . 1 settings.json

O workspace foi configurado para priorizar:

- LF;
- newline final;
- remoção de espaços finais;
- format on save;
- régua em 1 0 0 colunas;
- Prettier;
- ações do ESLint;

- organização de imports;
- atualização de imports ao mover arquivos;
- exclusão de `node_modules`, `.next`, coverage e client gerado das buscas;
- integração com instruções/agentes.

Preferências pessoais como tema, cor, fonte e zoom não devem ser impostas pelo repositório.

2 6 . 2 Extensões recomendadas

Entre as recomendações:

- ESLint;
- Prettier;
- Tailwind CSS;
- Prisma;
- Docker;
- GitLens;
- Error Lens;
- Markdownlint;
- EditorConfig;
- GitHub Copilot;
- Copilot Chat.

2 6 . 3 Tasks

Atalhos disponíveis incluem:

```
npm: dev
npm: lint
npm: typecheck
npm: test
npm: build
docker: up
docker: down
docker: logs
prisma: migrate dev
prisma: seed
prisma: studio
```

No VS Code:

```
Ctrl + Shift + P
→ Tasks: Run Task
```

2 7 . Snippets

Arquivo:

```
.vscode/soeirotech.code-snippets
```

stamp

Insere o carimbo digital de documentação.

adr

Cria estrutura de Architecture Decision Record.

devlog

Cria registro de desenvolvimento orientado a problema, decisão, validação e próximo passo.

nsc

Base de componente Server Component.

zschema

Base para schema Zod e tipo inferido.

vtest

Base de teste Vitest com Arrange, Act e Assert.

Regra

Snippets são aceleradores de escrita, não substitutos de entendimento.

2 8 . Agentes de IA

O projeto possui quatro agentes com responsabilidades separadas.

2 8 . 1 Arquiteto

Usar antes de grandes mudanças.

Perguntas que deve responder:

- qual é o problema real?
- quais módulos serão afetados?
- quais riscos existem?
- qual é a menor arquitetura suficiente?
- quais trade-offs existem?
- o que não precisa ser criado agora?

2 8 . 2 Implementador

Executa escopo aprovado.

Regras:

- inspecionar código existente;
- preferir diffs pequenos;
- não adicionar dependência sem necessidade;
- preservar tipagem;
- atualizar testes;
- informar validações e pendências.

2 8 . 3 Revisor

Prioriza:

- 1 . bugs;
- 2 . segurança;
- 3 . perda de dados/transações;
- 4 . regressões;
- 5 . testes;
- 6 . acoplamento e manutenção;
- 7 . performance quando plausível.

2 8 . 4 Mentor

Transforma o código real em aprendizado.

Deve explicar:

- problema;
- solução;
- conceito aplicado;
- alternativa relevante;
- trade-off;
- validação.

O objetivo é impedir que IA vire apenas geradora de código sem compreensão.

2 9 . Prompts reutilizáveis

Prompts do workspace:

```
/planejar-feature  
/revisar-mudanca  
/criar-adr  
/explicar-mudanca
```

Exemplo de fluxo

```
/planejar-feature  
"Criar listagem de projetos"  
↓  
plano  
↓  
Implementador  
↓  
código  
↓  
/revisar-mudanca  
↓  
correções  
↓  
/explicar-mudanca  
↓  
aprendizado
```

O `AGENTS.md` é a regra compartilhada do projeto para agentes.

Entre os princípios mais importantes:

- resolver problema antes de escolher tecnologia;
- evitar overengineering;
- não inventar requisitos;
- não trocar stack sem justificativa;
- não esconder erros com `any`;
- usar migrations;
- testar regras críticas;
- registrar decisões importantes;
- produzir evidência profissional;
- finalizar mudanças com validação.

Definition of Done

Uma tarefa só está concluída quando, conforme aplicável:

- requisito atendido;
- sem código temporário inexplicado;
- lint passa;
- typecheck passa;
- testes passam;
- build passa;
- documentação necessária atualizada;
- evidência registrada.

3 1 • Documentação oficial

`docs/00-product/`

Visão, problema, objetivos e escopo.

docs/01-requirements/

Requisitos funcionais e não funcionais.

docs/02-architecture/

Arquitetura e contexto.

docs/03-data/

Modelo de dados.

docs/04-decisions/

ADRs.

docs/05-quality/

Estratégia de testes e qualidade.

docs/06-project/

Roadmap e riscos.

docs/07-devlog/

Registros relevantes do desenvolvimento.

docs/08-showcase/

Case study e material profissional.

docs/reference/

Referências que ajudam a pensar, mas não definem automaticamente requisitos.

3 2 . Sistema de estudo

Estrutura recomendada:

```
study/
├─ 00-inbox/
├─ 01-concepts/
│   ├─ architecture/
│   ├─ docker/
│   ├─ nextjs/
│   ├─ postgresql/
│   ├─ prisma/
│   ├─ testing/
│   └─ typescript/
├─ 02-labs/
├─ 03-debugging/
├─ 04-reviews/
├─ 05-glossary/
├─ 06-applied-learning/
└─ templates/
```

3 2 . 1 00-inbox

Notas rápidas ainda não classificadas.

3 2 . 2 01-concepts

Conceitos consolidados com suas próprias palavras.

3 2 . 3 02-labs

Experimentos pequenos para responder perguntas técnicas.

3 2 . 4 03-debugging

Erros reais, hipóteses, causa raiz e correção.

3 2 . 5 04-reviews

Revisões periódicas do que foi aprendido.

3 2 . 6 05-glossary

Termos técnicos explicados de modo curto.

3 2 . 7 06-applied-learning

É uma das pastas mais valiosas. Registra:

```
conceito
↓
problema real
↓
onde foi aplicado
↓
trade-off
↓
validação
↓
evidência
```

3 3 . Como estudar sem transformar o projeto em apostila

Errado

```
Pós
├─ Disciplina
│   └─ Aula 01
│       └─ copiar resumo inteiro
```

Correto

```
Aprendi um conceito
↓
Entendi o problema que resolve
↓
Encontrei ou não uma necessidade no projeto
↓
Se necessário: apliquei
↓
Testei
↓
Registrei evidência
```

Regra de ouro

Você não precisa aplicar todo conceito estudado.

Saber dizer:

“Esse padrão não é necessário aqui porque acrescentaria complexidade sem benefício.”

É uma decisão de engenharia válida.

3 4 . Fluxo de desenvolvimento profissional

Para cada feature:

1. Problema
↓
2. Requisito
↓
3. Modelagem
↓
4. Plano mínimo
↓
5. Implementação
↓
6. Testes
↓
7. Revisão
↓
8. Documentação
↓
9. Evidência
↓
10. GitHub / Showcase quando aplicável

3 4 . 1 Limite de escopo

Uma feature deve ser pequena o bastante para ser explicada, testada e concluída.

Exemplo para **Project** :

- Etapa A - listar projetos
- Etapa B - criar projeto
- Etapa C - visualizar detalhe
- Etapa D - editar
- Etapa E - arquivar
- Etapa F - filtros

Isso é melhor do que tentar fazer “CRUD completo + autenticação + dashboard” numa única entrega.

3 5 . Git e GitHub

3 5 . 1 Estratégia de visibilidade

É válido manter o código completo em repositório privado e publicar uma amostra/showcase separada.

Estratégia proposta:

```
PRIVADO
soeirotech-engineering-atlas
→ código completo
→ histórico real
→ documentação interna

PÚBLICO
soeirotech-engineering-atlas-showcase
→ descrição do problema
→ arquitetura
→ screenshots
→ decisões
→ stack
→ resultados
→ link da demo
```

A rota `/showcase` já existe como base da apresentação pública.

3 5 . 2 O que mostrar publicamente

Um bom showcase deve mostrar:

- problema;
- solução;
- arquitetura;
- stack;
- decisões relevantes;
- screenshots;
- testes;
- deploy;
- desafios e trade-offs;
- aprendizados aplicados.

Não precisa expor dados pessoais, anotações privadas ou todo o histórico de estudo.

3 5 . 3 Commits

Commits devem representar mudanças compreensíveis.

Exemplos:

```
feat(projects): add project listing
fix(docker): isolate node_modules volume
refactor(prisma): centralize prisma client
test(projects): cover slug normalization
docs(adr): record modular monolith decision
```

3 6 . Segurança

3 6 . 1 Segredos

Nunca commitar:

- `.env` real;
- senhas reais;
- tokens;
- chaves de API;
- cookies;
- credenciais de deploy.

O repositório deve fornecer apenas `.env.example`.

3 6 . 2 Autorização

Quando autenticação for adicionada, autorização deve ser aplicada no servidor.

Esconder botão no frontend não é controle de acesso suficiente.

3 6 . 3 Validação

Entradas do usuário devem ser validadas no limite do sistema, futuramente com Zod conforme planejado.

3 6 . 4 Dados do showcase

O showcase público deve possuir uma fronteira explícita entre:

conteúdo apresentável
vs.
dados privados

3 7 . Riscos do projeto

Risco	Impacto	Mitigaçã
Virar cópia de aulas	Alto	Exigir problema real antes da feature
Overengineering	Alto	Modular monolith + ADR
Trocar stack constantemente	Alto	Manter stack-base estável
Acumular features	Alto	Trabalhar por fases pequenas
Expor dados privados	Alto	Separar showcase público
Docs desatualizados	Médio	Atualizar junto com mudanças relevantes

3 8 . Troubleshooting - Prisma não encontrado

Sintoma

```
sh: prisma: not found
```

Significado

O CLI Prisma não está disponível em `node_modules/.bin` dentro do container.

Causas possíveis

- dependências não instaladas;
- volume de `node_modules` incompleto;
- instalação interrompida;

- container criado com estado antigo.

Diagnóstico

```
docker compose exec web npx prisma -v
```

Se o serviço nem estiver rodando, primeiro resolver o container.

3 9 . Troubleshooting - `npm ENOEMPTY`

Sintoma encontrado

```
npm error ENOEMPTY: directory not empty, rmdir ...
```

O erro ocorreu dentro do volume de `node_modules` .

Abordagem segura

- 1 . identificar que o problema é o volume de dependências;
- 2 . não apagar `pgdata` ;
- 3 . remover/recriar somente o container e volume de `node_modules` quando necessário.

Volumes relevantes:

```
engineering_atlas_node_modules → dependências  
engineering_atlas_pgdata      → BANCO
```

O segundo contém dados persistidos. Não apagar por tentativa.

4 0 . Troubleshooting - Docker credential helper

Sintoma encontrado

```
failed to solve: error getting credentials - err: exit status 1
```

Também aparecia ao executar:

```
docker pull node:24-alpine
```

Diagnóstico que isolou o problema

Foi usada uma configuração Docker temporária sem credential helper:

```
mkdir -p ~/.docker-test  
printf '{"auths":{}}\n' > ~/.docker-test/config.json  
DOCKER_CONFIG=$HOME/.docker-test docker pull node:24-alpine
```

O pull funcionou. Isso demonstrou que:

Docker Engine	OK
Internet	OK
Docker Hub	OK
Imagem Node	OK
Credential helper	PROBLEMA

Aprendizado de debugging

Um erro durante `docker build` não significa automaticamente que o `Dockerfile` está errado.

Investigue em camadas:

```
Docker CLI  
→ credenciais  
→ registry  
→ imagem base  
→ build  
→ container  
→ aplicação
```

4 1 . Troubleshooting - porta ocupada

Situação

Outro projeto já utilizava a porta `3 0 0 0` no host.

Solução do Atlas

```
3010 -> 3000
```

A porta interna `3 0 0 0` pode continuar sendo usada porque o isolamento é feito pelo container.

Verificar

```
docker compose ps
```

`.env`

Confirmar:

```
WEB_PORT=3010
POSTGRES_PORT=5440
```

4 2 . Troubleshooting - `service web is not running`

Essa mensagem é consequência, não causa raiz.

Se `docker compose exec web ...` retorna:

```
service "web" is not running
```

investigue:

```
docker compose ps
docker compose logs web
```

Depois examine o erro de build/startup que impediu o serviço de ficar ativo.

4 3 . Troubleshooting - health degradado

Se `/api/health` retornar `5 0 3/degraded`:

- 1 . confirmar `db` saudável:

```
docker compose ps
```

- 1 . logs do banco:

```
docker compose logs db
```

- 1 . conferir `DATABASE_URL` ;
- 2 . confirmar hostname `db` dentro do Docker;
- 3 . verificar Prisma Client;
- 4 . testar conexão com PostgreSQL.

4 4 . Acesso direto ao PostgreSQL

Para entrar via `psql` dentro do container:

```
docker compose exec db psql -U atlas -d engineering_atlas
```

Dentro do `psql`:

```
SELECT id, name, slug, status FROM "Project";
```

Sair:

```
\q
```

Esse recurso é útil para aprender a diferença entre:

```
model Prisma  
vs.  
tabela PostgreSQL
```

4 5 . Roadmap

Fase 0 - Fundação

Status: preparada/concluída como base.

- Next.js;
- Docker;
- PostgreSQL;
- Prisma;
- health check;
- qualidade;
- docs;
- agentes/prompts.

Fase 1 - Projetos

- CRUD de projetos;
- validação;
- testes;
- filtros por status.

Fase 2 - Skills

- CRUD;
- relação projeto-skill;
- contexto de uso.

Fase 3 - Evidências

- cadastro;
- projeto obrigatório;
- skill opcional;
- filtros por tipo.

Fase 4 - Decisões

- ADRs no sistema;
- status e histórico.

Fase 5 - Showcase

- conteúdo público selecionado;
- projetos;
- skills;
- evidências.

4 6 . Ponto atual do aprendizado

A sequência atual foi organizada assim:

```
Prisma
|
├─ 00 - 0 que é Prisma
├─ 01 - model Project
├─ 02 - Migration          ← próximo estudo
├─ 03 - Prisma Client
├─ 04 - Primeiro SELECT
└─ 05 - Página /projects
```

O objetivo é não executar comandos mecanicamente. Antes de `migrate`, entender:

- o que é migration;
- por que não alterar banco manualmente sem rastreabilidade;
- o que o Prisma gera;
- onde o SQL fica;
- como migrations entram no Git.

4 7 . Plano de aprendizado por tecnologia

Prisma

- 1 . schema;
- 2 . model;
- 3 . tipos;
- 4 . annotations;
- 5 . enums;
- 6 . migrations;
- 7 . Client;
- 8 . queries;
- 9 . relations;
- 1 0 . transactions quando houver necessidade.

PostgreSQL

- 1 . tabela;
- 2 . coluna;
- 3 . primary key;
- 4 . unique;
- 5 . foreign key;
- 6 . index;
- 7 . constraints;
- 8 . joins;
- 9 . transações;
- 1 0 . análise de consultas quando houver necessidade.

Next.js

- 1 . App Router;
- 2 . page;
- 3 . layout;
- 4 . Server Components;
- 5 . Route Handlers;

- 6 . formulários/ações;
- 7 . cache/revalidation quando necessário;
- 8 . autenticação quando o domínio exigir.

TypeScript

- 1 . tipos básicos;
- 2 . objetos;
- 3 . unions/enums quando apropriado;
- 4 . inferência;
- 5 . tipos derivados de Zod/Prisma;
- 6 . generics quando houver problema real;
- 7 . evitar `any` .

Docker

- 1 . imagem;
- 2 . container;
- 3 . porta host/container;
- 4 . volume;
- 5 . network;
- 6 . healthcheck;
- 7 . build stages;
- 8 . produção somente depois de dominar desenvolvimento.

4 8 . Rotina de trabalho recomendada

Ao iniciar uma sessão

```
docker compose ps
```

Se necessário:

```
docker compose up -d
```

Depois:

- 1 . escolher uma tarefa;
- 2 . ler requisito/documento relacionado;
- 3 . identificar conceito que precisa entender;
- 4 . implementar o menor incremento.

Durante

- não trocar de feature no meio;
- registrar dúvida em `study/00-inbox`;
- criar debugging note para erros que ensinaram algo importante;
- evitar editar muitos arquivos sem necessidade.

Antes de encerrar

Conforme aplicável:

```
npm run lint
npm run typecheck
npm test
npm run build
```

Depois registrar:

- o que mudou;
- o que aprendeu;
- evidência;
- próximo passo único.

4 9 . Checklist antes de criar uma feature

☐

Sei qual problema estou resolvendo.

☐

Existe requisito ou necessidade identificada.

☐

Sei quais dados entram e saem.

☐

Sei qual é a menor entrega possível.

☐

Não estou adicionando tecnologia apenas para estudar.

☐

Sei como validar a mudança.

☐

Sei qual evidência pode ser produzida.

☐

Sei o que ficará fora do escopo.

5 0 . Checklist de mudança de banco

☐

Entendi a regra de domínio.

☐

Modelei no `schema.prisma`.

☐

Revisei campos obrigatórios/opcionais.

☐

Revisei `@unique`, relações e defaults.

☐

Criei migration com nome descritivo.

☐

Li o SQL gerado quando a mudança é relevante.

☐

Testei a migration.

☐

Atualizei seed se necessário.

☐

Atualizei documentação de dados se necessário.

☐

Não usei `db.push` para esconder uma migration que deveria ser rastreada.

5 1 . Checklist de Pull Request

☐

Objetivo descrito.

☐

Mudanças listadas.

☐

Lint executado.

☐

Typecheck executado.

☐

Testes executados.

☐

Build executado quando aplicável.

☐

Evidências incluídas.

☐

Riscos/trade-offs declarados.

☐

ADR incluído se a mudança for arquitetural.

5 2 . Checklist de evidência profissional

Uma feature importante deve tentar gerar pelo menos uma evidência:

☐

commit bem descrito;

☐

pull request;

☐

teste automatizado;

☐

ADR;

☐

diagrama;

☐

screenshot;

☐

release;

☐

deploy;

☐

benchmark, somente quando fizer sentido.

5 3 . Como transformar o projeto em case para emprego

O case não deve ser apresentado como:

“Fiz um CRUD com Next.js.”

Uma narrativa melhor é:

“Projetei uma plataforma para conectar habilidades a evidências reais de projetos. Comecei com um modular monolith em Next.js e PostgreSQL para reduzir complexidade, modeliei projetos, skills, evidências e decisões, automatizei qualidade com testes e CI e documentei decisões por ADR.”

Estrutura de apresentação

1. Problema
2. Usuário/necessidade
3. Escopo
4. Arquitetura
5. Modelo de dados
6. Decisões
7. Implementação
8. Testes
9. Desafios
10. Trade-offs
11. Resultado
12. Próximos passos

5 4 . O que não prometer em entrevista

Enquanto ainda não estiver implementado/validado, não afirmar que o projeto possui:

- alta disponibilidade real;
- escalabilidade comprovada;
- observabilidade completa;

- segurança de produção auditada;
- backup/restore validado;
- deploy automatizado completo;
- microserviços;
- autenticação robusta;
- cobertura de testes abrangente.

É profissional diferenciar:

implementado
planejado
estudado
considerado

5 5 . Melhorias técnicas recomendadas para a fundação

Estas melhorias são recomendações para fases futuras e devem ser feitas uma de cada vez:

- 1 . gerar e versionar `package-lock.json` ;
- 2 . remover `npm install` do startup do container após estabilizar a estratégia de dependências;
- 3 . validar o estágio `production` do Dockerfile;
- 4 . revisar seed para torná-lo totalmente idempotente, inclusive evidências;
- 5 . configurar Prisma Studio apenas se houver utilidade real;
- 6 . implementar primeira feature `/projects` ;
- 7 . introduzir Zod quando houver entrada real de formulário;
- 8 . adicionar testes de integração quando surgirem regras que justificarem;
- 9 . definir autenticação apenas quando existir necessidade de dados privados multiusuário ou controle de acesso.

5 6 . Glossário

ADR

Architecture Decision Record. Documento curto que registra contexto, decisão, alternativas e consequências.

API

Interface pela qual sistemas ou partes do sistema se comunicam.

Build

Processo que transforma/verifica o projeto para execução/distribuição.

CI

Continuous Integration. Execução automática de verificações em commits/PRs.

Container

Processo isolado criado a partir de uma imagem Docker.

CUID

Tipo de identificador gerado usado no schema atual como ID.

Docker Compose

Arquivo/ferramenta para declarar e executar múltiplos serviços relacionados.

E 2 E

End-to-end. Teste que percorre um fluxo próximo ao comportamento real do usuário.

Enum

Conjunto fechado de valores permitidos.

Foreign Key

Chave que relaciona registro de uma tabela a outra.

Health Check

Verificação automatizada de saúde de um serviço.

Índice

Estrutura do banco que pode acelerar determinadas consultas, com custo de espaço/escrita.

Migration

Mudança versionada na estrutura do banco.

Modular Monolith

Aplicação implantada como unidade, mas organizada internamente em módulos e fronteiras claras.

ORM

Ferramenta/camada que auxilia o mapeamento e acesso a dados relacionais. Prisma cumpre esse papel de forma própria no projeto.

Primary Key

Identificador Único principal de uma tabela.

Prisma Client

Cliente TypeScript gerado para acessar os modelos do banco.

Route Handler

Forma do Next.js App Router de implementar endpoints HTTP.

Seed

Dados iniciais/de desenvolvimento inseridos no banco.

Server Component

Componente React executado no servidor por padrão no App Router, quando não requer interatividade de cliente.

Slug

Texto amigável e Único usado em URLs/identificadores legíveis.

Typecheck

Verificação de tipos pelo TypeScript sem emitir build.

Volume

Armazenamento persistente gerenciado pelo Docker.

5 7 . Referência rápida de comandos

Docker

```
docker compose ps
docker compose up -d
docker compose up --build -d
docker compose logs -f web
docker compose logs -f db
docker compose stop
docker compose down
```

Prisma dentro do container

```
docker compose exec web npx prisma -v
docker compose exec web npm run db:generate
docker compose exec web npm run db:migrate -- --name init
docker compose exec web npm run db:seed
```

Qualidade

```
docker compose exec web npm run lint
docker compose exec web npm run typecheck
docker compose exec web npm test
docker compose exec web npm run build
```

PostgreSQL

```
docker compose exec db psql -U atlas -d engineering_atlas
```

Git - fluxo básico

```
git status
git diff
git add <arquivos>
git commit -m "tipo(escopo): descrição"
git push
```

5 8 . Ficha rápida do projeto

Item	Valor
Projeto	SoeiroTech Engineering Atlas
Versão	0 . 1 . 0
Arquitetura	Modular Monolith
Linguagem	TypeScript
Framework	Next.js
UI	React
Banco	PostgreSQL 1 7 Alpine
ORM/Data Toolkit	Prisma 7 . 9 . 1
Container	Docker Compose
Web host	3 0 1 0
DB host	5 4 4 0
Web container	3 0 0 0
DB container	5 4 3 2
Teste unitário	Vitest
E 2 E	Playwright
CI	GitHub Actions
Documentação oficial	docs/

Item	Valor
Aprendizado	<code>study/</code>
Primeira feature	Projects

5 9 . Próxima etapa oficial

O ponto recomendado para continuar é:

```
study/01-concepts/prisma/  
02-migrations.md
```

Antes de executar novas migrations por hábito, estudar:

- 1 . o que é migration;
- 2 . por que ela é versionada;
- 3 . o que o Prisma gera;
- 4 . como o SQL chega ao PostgreSQL;
- 5 . o que deve ser commitado;
- 6 . diferença entre alteração de schema e alteração de dados.

Depois disso, a sequência é:

```
Migration  
↓  
Prisma Client  
↓  
primeira consulta real  
↓  
/projects
```

6 0 . Encerramento

O Engineering Atlas deve crescer devagar o suficiente para que cada parte seja compreendida e rápido o suficiente para produzir entregas reais.

A medida de progresso não é a quantidade de frameworks presentes no repositório. É a capacidade de explicar:

- qual problema existia;
- por que determinada solução foi escolhida;
- como foi implementada;
- como foi validada;
- qual trade-off foi aceito;
- que evidência demonstra o resultado.

O princípio do projeto permanece:

Estudo fornece conhecimento. Projeto fornece contexto. Evidência demonstra competência.

Anexo A - Carimbo Digital

Modelo recomendado para ADRs, DevLogs, relatórios de teste e registros relevantes:

 Painel de Sinais Organizados (Compacto)
 [PJ]  [DEV]  [STUDY]  [CODE]  [TEST]  [DOC]

Projeto: SoeiroTech Engineering Atlas

Artefato: <nome>

Status: Planejado | Em desenvolvimento | Em revisão | Concluído

Objetivo: <uma frase>

Evidência: <commit, PR, teste, screenshot, release ou deploy>

Não inserir esse carimbo dentro de arquivos de código-fonte.

Anexo B - Modelo de ADR

ADR-000 - Título

- Status: Proposto
- Data: AAAA-MM-DD

Contexto

Qual problema exige uma decisão?

Decisão

O que foi decidido?

Alternativas consideradas

- Alternativa A
- Alternativa B

Consequências positivas

- Benefício

Consequências negativas / trade-offs

- Custo ou limitação

Evidências

- Issue
- PR
- Commit
- Teste/benchmark

Anexo C - Modelo de aprendizado aplicado

```
# Aprendizado Aplicado - <conceito>

## Conceito estudado

## Problema real do projeto

## Como foi aplicado

## Arquivos envolvidos

## Por que esta solução?

## Trade-off

## Como foi validado?
- [ ] lint
- [ ] typecheck
- [ ] testes
- [ ] build
- [ ] teste manual

## Evidências
- Commit
- PR
- ADR
- Screenshot
```

Anexo D - Regra visual de evolução

```
ESTUDO
  ↓
conceito entendido
  ↓
PROBLEMA REAL
  ↓
requisito
  ↓
IMPLEMENTAÇÃO
  ↓
teste
  ↓
REVISÃO
  ↓
evidência
  ↓
GITHUB / SHOWCASE
```